



FS



← Découvrez les librairies Python pour la Data Science

13% de progression

Filtrez les données du data frame



Imaginons les situations suivantes :

- pour une offre commerciale spécifique, on cherche à ne sélectionner que les hommes de notre data frame ;
- on souhaite identifier les hauts revenus parmi notre clientèle. Pour cela, on souhaite sélectionner tout ceux gagnant mensuellement plus de x € ;
- on souhaite identifier les clients dits "risqués", c'est-à-dire ceux qui ont un taux d'endettement supérieur aux 35 % légaux.

Jusqu'à présent, nous avons été capables de sélectionner des colonnes spécifiques et de les manipuler à notre guise. Mais les exemples précédents ne peuvent être satisfaits avec de simples manipulations de colonnes : il faut appliquer une condition spécifique pour ne sélectionner que les lignes pertinentes.

Cette opération est ce qu'on appelle une **restriction**, ou un **filtrage**. Naturellement, Pandas permet de faire cela, pour avoir un contrôle total sur nos données. Voyons cela ensemble !

Appliquez la sélection via des indices

Le premier type de sélection est réalisé via des indices. Il se fait via la méthode `.iloc` .

Pour rappel, un indice est la position à laquelle se trouve un élément au sein d'un tableau, que ce soit une liste, une Series ou un array. En Python, les indices commencent à 0 : ainsi, le premier élément aura pour indice 0, le second 1, etc.



La méthode `iloc` suit la syntaxe suivante :

```
mon_dataframe.iloc[indice_ligne, indice_colonne] .
```

Prenons un exemple concret : on souhaite sélectionner le nom du premier client de notre base de données. Comme c'est le premier client, **son indice de ligne sera 0**. Nous souhaitons sélectionner son nom ; la variable nom est la 3e variable de notre data frame, l'**indice de sa colonne sera donc... 2**. Eh oui ! Pour les colonnes aussi, les indices commencent à 0. Le code de sélection serait donc :

```
clients.iloc[0,2]
```

```
'Laurent Dagenais'
```

Exemple de sélection via un indice

Cela correspond bien au nom de notre premier client.

Maintenant, admettons que je souhaite sélectionner le nom des 10 premiers clients, comment procéder ?

Comme avec les arrays, nous pouvons utiliser l'opérateur `:` pour sélectionner la plage souhaitée ! Voici quelques exemples :

```
1 # sélectionner le nom des 10 premiers clients :  
2 clients.iloc[:10, 2]
```

```
1 # sélectionner le genre des 10 premiers clients :  
2 clients.iloc[:10, 3]
```



```
1 # sélectionner toutes les colonnes des 10 premiers clients :  
2 clients.iloc[:10, :]
```

```
1 # sélectionner l'email et le nom des 10 clients suivants :  
2 clients.iloc[10:20, 1:3]
```

```
1 # sélectionner l'e-mail et le genre des 10 derniers clients :  
2 clients.iloc[-10:, [1, 3]]
```

La méthode `iloc` peut être utile dans certains cas, mais elle est finalement assez limitée. En effet, elle n'est pas adaptée si on souhaite



revenus supérieurs à x €, et il faut absolument connaître l'indice des différentes colonnes. Cela reste jouable avec 5-6 colonnes, mais devient beaucoup plus compliqué lorsqu'on est en présence de dizaines de colonnes.

Il existe une autre méthode, qui est similaire à `iloc` mais qui permet de pallier l'ensemble de ces problématiques : la méthode `.loc`.

Utilisez la sélection via des conditions

Pour être tout à fait honnête avec vous, les data frames possèdent de nombreuses méthodes, comme vous avez pu commencer à vous en rendre compte. Mais de toutes, il y en a certainement une que vous allez utiliser et réutiliser à outrance : c'est la méthode `.loc` !

Celle-ci suit la syntaxe suivante :

```
mon_dataframe.loc[ condition sur les lignes,  
colonne(s) ]
```

C'est à première vue très simple, mais nous allons voir que cette syntaxe peut en pratique devenir vite complexe, et peut permettre en même temps une grande flexibilité. C'est **LA** méthode qui pourra répondre à près de 90 % de vos besoins en termes de sélection d'une partie de vos données. La source de ce chiffre ? OK, c'est un avis totalement personnel ! Mais laissez-moi vous montrer pourquoi je suis aussi sûr de moi.



Une autre option de la méthode `.loc` est la sélection par **index**. Avant d'aller plus loin sur cet aspect, laissez-moi redéfinir la différence entre des index et des indices ! Les indices, comme nous l'avons vu plus haut, sont la position intrinsèque d'un élément au sein d'un tableau. Les index, en revanche, correspondent à une valeur qui est associée à chaque ligne. C'est ce que vous voyez sur la gauche de votre data frame :

```
clients.head()
```

	identifiant	email	nom	genre
0	0	LaurentDagenais@rhyta.com	Laurent Dagenais	M
1	1	GuyMarois@fleckens.hu	Guy Marois	M
2	2	BeaufortLesage@einrot.com	Beaufort Lesage	M
3	3	RussellDurand@armyspy.com	Russell Durand	M
4	4	AlexisRiel@rhyta.com	Alexis Riel	M

Index

Localisation de l'index dans le data frame

Par défaut, ils correspondent... aux indices ! Mais ils peuvent ne pas être numériques ; on pourrait par exemple fixer le nom de la personne comme index, si on est sûr qu'il n'y aura pas de doublons.

Faisons un simple tri pour bien comprendre la différence entre l'indice et l'index :

```
clients.sort_values('nom').head()
```



	identifiant		email	nom	genre
13	13		AcelineHughes@jourrapide.com	Aceline Hughes	F
15	15		AgateGrandbois@gustr.com	Agate Grandbois	F
7	7		AgramantPepin@dayrep.com	Agramant Pepin	M
16	16		AgricanFoucalt@jourrapide.com	Agrican Foucalt	M

Différence entre index et indice

Cette situation est intéressante, car si on regarde notre première ligne, son index est 13. Pourtant, c'est bel et bien la première ligne, donc son indice est 0 !!!

Je pense que vous commencez à comprendre la différence : un indice est relatif aux opérations réalisées. Un individu peut avoir le premier indice à un moment, et un tout autre indice en fonction du tri effectué. En revanche, l'index est intrinsèque aux lignes au sein d'un data frame. Peu importe les opérations effectuées, que ce soit un tri, une suppression, etc., **l'index sera toujours le même** ! La seule condition, c'est que chaque ligne doit avoir un index unique : c'est-à-dire qu'il ne doit pas y avoir deux fois la même valeur au sein d'un index ; on ne pourra par exemple pas avoir 2 fois l'index 0, etc.

Pourquoi je vous parle de tout cela ?

Parce qu'il est possible de faire une sélection au sein d'un data frame selon les index, avec la méthode `.loc`.

Faites le test avec les lignes suivantes :

```
1 display(clients.loc[0:10, :])
2 display(clients.iloc[0:10, :])
```

Il est d'ailleurs possible d'accéder à la liste des index d'un data frame via l'attribut `.index`. Par exemple, `clients.index` nous renverra l'ensemble des index de notre data frame.

Il est parfois intéressant, après avoir effectué certaines opérations, de réinitialiser ces index – nous verrons un exemple concret, dans la prochaine section, où cela est même nécessaire. C'est possible via la méthode `.reset_index()` :



```
1 df_temp = df.reset_index(drop=True)
2 display(df_temp)
```

```
1 # reset_index :
2 display(df_temp.reset_index())
```

```
1 # reset_index sans garder les anciens Index :
2 display(df_temp.reset_index(drop=True))
```

Modifiez une sélection

Comme avec les colonnes, on peut utiliser les méthodes `.iloc` ou `.loc` pour modifier un data frame existant. C'est même encore plus intéressant que ce que nous avons vu jusque-là, car il est possible de modifier précisément une partie d'un data frame. Je vous propose de



03:59

Que diriez-vous à présent de mettre tout cela en pratique ? C'est parti !

À vous de jouer



Contexte

Certaines demandes ont été spécifiquement formulées par le responsable du service Prêt de notre établissement. Il souhaite que vous puissiez appliquer les traitements nécessaires pour lui donner des réponses précises.

Consignes

Vous allez devoir mettre en application l'ensemble des processus de sélection présentés ci-dessus, pour répondre à ces différentes demandes :



communiquer le nombre de personnes ayant dépassé ce seuil ?

- Même question, mais cette fois-ci uniquement sur l'agence parisienne.
- Pour faciliter le traitement d'éventuelles futures demandes de prêts, pourriez-vous ajouter une variable nommée *risque*, qui nous permettrait d'identifier facilement les clients à risque ?
- Combien de prêts automobiles ont été accordés ? Quel est le coût total moyen de ceux-ci ?
- Quel est le bénéfice mensuel total réalisé par l'agence toulousaine ?

Vous êtes prêt ? C'est parti pour [l'exercice](#) !

Vérifiez votre travail

Une fois celui-ci terminé, je vous invite à vérifier vos réponses avec [la correction ci-contre](#).

En résumé

- Il existe deux méthodes pour sélectionner précisément des éléments au sein d'un data frame :
 - La méthode `.iloc` permet de sélectionner à partir des indices. La syntaxe est :

```
mon_dataframe.iloc[ indice(s) ligne ,  
indice(s) colonne ]
```
 - La méthode `.loc` permet de sélectionner à partir de conditions et des noms de colonnes. La syntaxe est :

```
mon_dataframe.loc[ condition sur les lignes  
, colonne(s) ]
```

- La condition sur les lignes est une condition qui va être testée sur chaque ligne. Une ligne est conservée dans le processus de sélection si elle satisfait à cette condition. Cette condition peut



- Les index sont des valeurs qui sont associées intrinsèquement à chaque ligne. Si on effectue un tri ou toute autre opération, une ligne, quelle que soit sa position dans le data frame, aura toujours le même index. Il est possible de réinitialiser ce dernier via la méthode `.reset_index`.
- La méthode `.loc` peut être également utilisée pour modifier la partie du data frame sélectionnée.

Maintenant que nous sommes au point sur la sélection avec Pandas, que diriez-vous d'aller encore un peu plus loin dans la manipulation de données avec Pandas, en découvrant comment faire une agrégation ?



[Avez-vous une suggestion pour nous ?](#) 

Chapitre suivant →